
MILESTONE 2 — EXECUTION DOCUMENT

FreightQuote AI

Pricing Intelligence & Margin Optimisation — Weeks 3–4 Build Specification

DOCUMENT TYPE	Milestone execution spec — cost model, incoterms, margin policy, approvals, APIs, day plan
PROJECT CODE	FQ-AMB-001 / M2
DURATION	10 working days (Weeks 3–4)
EXIT CRITERIA	Cost deviation $\leq$ 8% vs 150 historical quotes • Zero margin-floor violations • Quote generated in < 60s p95
PREREQUISITE	Milestone 1 signed off: routes, transit, gateways and distances all working
CORE CHANGE	The indicative 35% uplift is deleted. Every rupee becomes a named, defensible line item.

Table of Contents

1. What Milestone 2 Must Deliver

Scope boundary and exit criteria

2. The Cost Model

Every charge, how it is calculated, in what order

3. Incoterm Cost Responsibility

Which legs belong in the quote

4. Rate Cards

Structure, weight breaks, import and validation

5. Margin Policy & Approval Rules

Floors, resolution order, approver routing

6. Quote Versioning

Immutability mechanics and what gets frozen

7. Calculation Method Specification

Every method: input, output, formula, module

8. Data Model Changes

New collections and what changes in existing ones

9. Frontend Build List

Pages, components, connection notes

10. Backend Build List

Modules and endpoints

11. Day-by-Day Plan

10 working days

12. Definition of Done

Signed off before Milestone 3 starts

1. What Milestone 2 Must Deliver

1.1 The change in one picture

MILESTONE 1 (what exists now)	MILESTONE 2 (what you are building)		
lane_rate.per_container × 2 = ₹2,84,814 × 1.35 flat uplift = ₹3,84,500 "indicative"	Ocean freight	2 × 40HC	₹2,84,814
	BAF (fuel) 12%		₹34,178
	THC origin	2 × ₹9,250	₹18,500
	THC destination	2 × ₹8,400	₹16,800
	ISPS security	2 × ₹2,100	₹4,200
	Documentation	per shipment	₹3,500
	TOTAL COST		₹3,61,992
	Margin @ 14.2%		₹59,905
	SELL PRICE		₹4,21,897

Note that the total moved. The placeholder was never correct — it was only plausible. Expect real prices to differ from Milestone 1 output by 5-15% on most lanes, and treat that as the system working, not breaking.

1.2 Scope boundary

IN Milestone 2	NOT in Milestone 2
Rate card model, import, validation, versioning	Weather risk assessment (M3)
Surcharge definitions and application engine	Customs validation and documentation checklists (M3)
Full cost build-up honouring incoterm	Composite shipment risk scoring (M3)
Air weight breaks and the lower-break rule	Multi-agent orchestration and fusion (M4)
Margin policy: floors, targets, resolution order	Executive dashboards and rollups (M4)
Approval rule engine and approver routing	Win-probability model beyond a simple baseline
Immutable quote versioning	Automated carrier rate API integration
Branded PDF quotation generation	Customer acceptance workflow beyond accept/decline
Customer portal quote view (sell price only)	Booking, tracking, invoicing
FX rate freezing per quote version	Multi-currency rate cards in more than one base

The scope trap in this milestone

Carrier rate APIs. Someone will suggest integrating directly with Maersk or CMA CGM instead of importing spreadsheets. Do not attempt it in M2. Carrier API access requires commercial agreements, credentials and per-carrier adapters, and the timeline for getting access is measured in weeks of paperwork you do not control. Build the file import path, which works regardless. API integration is a Phase 2 enhancement that plugs into the same rate card model.

1.3 Exit criteria and measurement

Criterion	Measurement	Evidence at review
Freight pricing engine operational	Reconcile against 150 historical quotes the brokerage actually sent	Report showing mean absolute deviation ≤ 8% on total cost, plus a per-component error breakdown
Margin optimisation functioning	Automated test across every lane × customer tier combination	Zero quotes below the applicable floor; every below-floor attempt produced a 409 and an approval record
Dynamic quote generation successful	End-to-end timing under 10 concurrent runs	p95 < 60 seconds measured at the gateway

On the 8% target: if your engine is 20% off against quotes your own brokers sent last quarter, the fault is almost always one of three things — wrong incoterm handling, a missing surcharge, or the wrong rate card being selected. The per-component breakdown tells you which. Do not tune the total to hit the number; fix the component.

2. The Cost Model

2.1 Calculation order matters

Surcharges that are a percentage of base must be computed after base. Insurance is a percentage of declared value *plus* freight, so it comes near the end. Run the build-up in this fixed order:

1. Base freight — from the rate card, by charge basis (per container / per kg / per R/T), with the minimum charge applied.
2. Freight surcharges — BAF, CAF, LSS, PSS, war risk. Percentage-of-base ones use step 1's result.
3. Origin charges — THC, ISPS, export documentation, customs clearance. Included only if the incoterm makes them the seller's cost.
4. Destination charges — THC, import documentation, delivery order fee. Incoterm-dependent.
5. Haulage — pickup and delivery legs. Incoterm-dependent. Base + per-km.
6. Special handling — hazmat, reefer, out-of-gauge. Percentage of base or flat.
7. Insurance — percentage of (declared value + freight + 10%), if requested.
8. Sum to COST — this is the buy side. Never shown to a customer.
9. Margin — resolved from policy, floor enforced, applied to cost.
10. SELL PRICE — cost + margin. This is what the quotation document shows.

2.2 Cost component catalogue

Code	Name	Calculation type	Notes
OFR	Ocean freight	PER_CONTAINER / PER_RT	Base rate from the rate card
AFR	Air freight	PER_KG with weight breaks	See section 4.3
BAF	Bunker adjustment factor	PERCENT_OF_BASE or PER_CONTAINER	Fuel. Varies monthly — must be dated.
CAF	Currency adjustment factor	PERCENT_OF_BASE	Applies when the rate card currency differs from the quote currency
LSS	Low sulphur surcharge	PER_CONTAINER	Emission-control area routes
PSS	Peak season surcharge	PER_CONTAINER	Date-windowed. Must respect valid_from/valid_to.
WRS	War risk surcharge	PER_CONTAINER	Route-conditional (Red Sea, Gulf)
THCO	Terminal handling — origin	PER_CONTAINER	Port-specific, not carrier-specific
THCD	Terminal handling — destination	PER_CONTAINER	Port-specific
ISPS	Security surcharge	PER_CONTAINER / PER_KG	Per-shipment on air
DOC	Documentation / B/L fee	FLAT_PER_SHIPMENT	One per shipment regardless of container count
CCO	Customs clearance — origin	FLAT_PER_SHIPMENT	Incoterm-dependent
CCD	Customs clearance — destination	FLAT_PER_SHIPMENT	Incoterm-dependent (DDP only)
PUH	Pickup haulage	BASE_PLUS_PER_KM	Uses the leg-1 road distance from M1
DLH	Delivery haulage	BASE_PLUS_PER_KM	Uses the leg-3 road distance
HAZ	Hazardous surcharge	PERCENT_OF_BASE (typ. 25%)	Triggered by is_hazardous
RFR	Reefer surcharge	PER_CONTAINER	Triggered by temperature-controlled
INS	Cargo insurance	PERCENT_OF_VALUE	Typically 0.3–0.5% of CIF value, with a minimum premium

2.3 Calculation types

FLAT_PER_SHIPMENT	amount	(once, regardless of quantity)
PER_CONTAINER	amount × container_count	
PER_KG	amount × chargeable_weight_kg	(min charge applies)
PER_RT	amount × revenue_tons	(min charge applies)
PERCENT_OF_BASE	base_freight × (pct / 100)	
PERCENT_OF_VALUE	(declared_value + freight) × 1.10 × (pct / 100)	
BASE_PLUS_PER_KM	flat + (road_km × per_km_rate)	

Every component carries a source label

Each line records whether it came from `RATE_CARD`, `SURCHARGE_TABLE`, `PREDICTED` or `MANUAL`. The broker must be able to see at a glance which numbers are firm and which are estimates. Any quote containing a `PREDICTED` component automatically requires approval — that is approval rule #4 in section 5.4.

3. Incoterm Cost Responsibility

You collected incoterm in Milestone 1 but nothing consumed it. Now it decides which legs belong in the quote at all. Same cargo, same route, different total — purely because of three letters. Get this table wrong and every quote is wrong.

3.1 The responsibility matrix

Shaded cells are the seller's cost, and therefore **in scope for the quote**.

Incoterm	Pickup haulage	Export customs	Origin THC	Main freight	Insurance	Dest THC	Import customs	Delivery haulage
EXW Ex Works	-	-	-	-	-	-	-	-
FCA Free Carrier	Yes	Yes	-	-	-	-	-	-
FOB Free On Board	Yes	Yes	Yes	-	-	-	-	-
CFR Cost & Freight	Yes	Yes	Yes	Yes	-	-	-	-
CIF Cost, Ins, Freight	Yes	Yes	Yes	Yes	Yes	-	-	-
DAP Delivered At Place	Yes	Yes	Yes	Yes	Yes	Yes	-	Yes
DDP Delivered Duty Paid	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

3.2 How this is implemented

```
INCOTERM_SCOPE = {
  "EXW": [],
  "FCA": ["PUH", "CCO"],
  "FOB": ["PUH", "CCO", "THCO", "ISPS", "DOC"],
  "CFR": ["PUH", "CCO", "THCO", "ISPS", "DOC", "OFR", "BAF", "CAF", "LSS", "PSS"],
  "CIF": [... CFR components ..., "INS"],
  "DAP": [... CIF components ..., "THCD", "DLH"],
  "DDP": [... DAP components ..., "CCD"]
}

build_cost_breakdown(shipment, route, rate_card):
  allowed = INCOTERM_SCOPE[shipment.service.incoterm]
  components = []
  for code in CALCULATION_ORDER:
    # section 2.1 order
    if code not in allowed: continue
    if not applies_to(code, shipment, route): continue
    components.append(calculate_component(code, ...))
  return components
```

EXW produces a quote with zero cost components — and that is correct

Under Ex Works the seller does nothing; the buyer arranges everything. Your engine must handle this without crashing and the UI must explain it rather than showing a blank table. Add EXW to the test suite explicitly — it is the edge case that breaks naive implementations.

3.3 Two rules that catch most errors

- Insurance under CIF has a formula defined by Incoterms rules:** minimum 110% of the invoice value. Do not use a plain percentage of declared value — use $(\text{declared_value} + \text{freight}) \times 1.10 \times \text{rate}$.
- DDP requires import duty, which you cannot calculate reliably in M2.** Duty depends on HS code, country, and trade agreements — that is Milestone 3 customs work. For M2, either exclude DDP from the incoterm dropdown, or include it with a mandatory manual duty entry and an approval requirement. Do not guess a duty figure.

4. Rate Cards

4.1 Structure

```
rate_cards          one document per carrier contract or spot sheet
├─ carrier_id, type (CONTRACT|SPOT|TARIFF)
├─ currency, valid_from, valid_to
└─ status (DRAFT|ACTIVE|EXPIRED|SUPERSEDED)

rate_card_lines      one document per lane × container type
├─ rate_card_id
├─ origin_port_code, destination_port_code, lane_key
├─ container_type (ocean) OR weight_break_min (air)
├─ base_rate { amount: Decimal128, currency }
└─ minimum_charge { amount, currency }

surcharges           independent of rate cards – port and route driven
├─ code, name, calculation_type, value, currency
├─ applies_to { modes[], ports[], lanes[], container_types[] }
└─ valid_from, valid_to
```

4.2 Rate card resolution — most specific wins

```
resolve_rate_card(shipment, route, customer):
  candidates = rate_card_lines matching lane + container_type
               where parent card is ACTIVE
               and valid_from <= ready_date <= valid_to

  priority order:
    1. CONTRACT card assigned to this specific customer
    2. CONTRACT card for this carrier (general)
    3. SPOT card for this carrier
    4. TARIFF card (published, worst rate)
    5. none found → market rate model → label PREDICTED

  return best match + which rule matched (stored on the quote)
```

Store **which rule matched** on the quote version. When a broker asks why one customer got a better rate, the answer must be one click away, not an investigation.

4.3 Air weight breaks and the lower-break rule

Air rates step down as weight increases. A rate card line for air carries a `weight_break_min`:

Weight break	Rate per kg	Applies when chargeable weight is
MIN	—	Minimum charge, e.g. ₹3,500 flat
+45	₹220	45–99 kg
+100	₹195	100–299 kg
+300	₹180	300–499 kg
+500	₹165	500–999 kg
+1000	₹148	1000 kg and above

The lower-break rule — brokers will notice if you skip this

At 280 kg the +100 break gives $280 \times ₹195 =$

₹54,600

. But charging at the next break's minimum weight gives $300 \times ₹180 =$

₹54,000

— cheaper, for more allowance. Real freight systems always check the next break up and use it when it produces a lower total. Implement it as: compute at the applicable break, compute at each higher break's minimum weight, take the lowest. Record which break was used on the component line.

4.4 Import: two calls, never one

1. Upload → validate. POST `/pricing/rate-cards/validate` parses the file and returns a report: rows parsed, rows rejected with reasons, unknown port codes, unknown container types, overlapping validity with an existing active card, currency mismatches. **Nothing is written to the database.**

2. Human reviews the report in the UI. Row-level errors are listed with line numbers.

3. Commit. POST `/pricing/rate-cards/commit` with the validation token. Inserts the card and lines in one transaction, supersedes any overlapping card.

Why this matters: one bad spreadsheet committed on upload silently corrupts every quote on those lanes until somebody notices — possibly after quotes have gone to customers. A validation step costs one extra click and prevents an entire class of incident.

Validation rules for the importer

Check	Action on failure
Port code exists in ports	Reject row, report the unknown code
Container type exists	Reject row
Rate is a positive number, parseable as Decimal	Reject row — scientific notation and currency symbols in spreadsheets are common
Currency is a known ISO code	Reject the whole file
<code>valid_from < valid_to</code>	Reject the whole file
No overlapping active card for the same carrier and lane	Warn, and require an explicit "supersede existing" confirmation
Duplicate lane + container rows within the file	Reject the whole file — ambiguous intent
Rate is more than 3× or less than 0.3× the previous card's rate	Warn — usually a decimal-place or unit error, occasionally a genuine market move

5. Margin Policy & Approval Rules

5.1 Policy structure

```
margin_policies
├── scope      GLOBAL | CARGO_TYPE | LANE | CUSTOMER_TIER | CUSTOMER_LANE
├── scope_key  e.g. "INNSA-AEJEA" or "STRATEGIC" or "STRATEGIC|INNSA-AEJEA"
├── floor_pct  never go below without approval
├── target_pct the default applied
├── stretch_pct aspirational, used by the Margin Agent as an upper anchor
└── effective_from, is_active
```

5.2 Resolution order — most specific wins

Order	Scope	Example
1	CUSTOMER_LANE	Sharma Textiles on INNSA-AEJEA: floor 9%, target 12%
2	CUSTOMER_TIER	STRATEGIC customers: floor 10%, target 13%
3	LANE	INNSA-AEJEA: floor 12%, target 15%
4	CARGO_TYPE	Hazardous: floor 18%, target 22%
5	GLOBAL	Fallback: floor 13%, target 16%

Return the first match. Store the resolved policy ID on the quote version so a later reviewer can see exactly which rule applied.

5.3 Where enforcement lives

```
Margin Agent (FastAPI)      → recommends 11.0%
Core API margin_policy.py   → resolves floor = 13.0%
                             RECOMMENDATION SUPPRESSED
                             applies 13.0%
                             records { recommended: 11.0,
                                       applied: 13.0,
                                       suppressed: true,
                                       floor_source: "LANE" }
```

The AI recommends; the backend decides

A language model or an optimiser may suggest any margin. The floor is a business rule enforced deterministically in Django/FastAPI service code, after the agent run, before the quote is persisted. Never let the agent's output be the last thing between a cost calculation and a customer price. The React slider showing a red warning is user experience; the `PATCH /margin` endpoint returning 409 is the control.

5.4 Approval rules

Order	Condition	Approver
1	Margin below floor by more than 5 percentage points	PRICING_MANAGER
2	Margin below floor by up to 5 percentage points	SENIOR_BROKER
3	Quote value above the high-value threshold	PRICING_MANAGER
4	Any component sourced as PREDICTED (no contract rate)	SENIOR_BROKER
5	New customer with no credit profile	SENIOR_BROKER
6	Rate card expires before the quote validity ends	SENIOR_BROKER

Rules evaluate in order and **all matching rules apply** — a quote can require two approvals. Rules are data, editable by Admin, and every evaluation writes to the audit log.

6. Quote Versioning

6.1 The rule

A quote version is written once and never updated. The only field ever modified afterwards is its status, and only to `SUPERSEDED`. A price change inserts a new version.

6.2 What gets frozen onto every version

Frozen field	Why
fx_rate + fx_rate_date	Rates move daily. A quote must reproduce identically in three months.
rate_card_id + resolution rule	Cards get superseded. You must know which one priced this quote and why it was chosen.
surcharge_ids[]	Surcharge values change monthly.
margin_policy_id + floor applied	Policies change. The applied floor must be evidenced.
route_snapshot	Routes get regenerated. A reference alone would let history be rewritten behind an issued quote.
model_versions	Which transit and market-rate models produced the estimates.
data_as_of	How old the inputs were at quote time.
cost_components[]	The full itemised build-up, not just the total.

6.3 Version transitions

v1 DRAFT	—edit margin—>	v2 DRAFT	(v1 → SUPERSEDED)
v2 DRAFT	—submit—>	v2 PENDING_APPROVAL	
v2 PENDING	—approve—>	v2 APPROVED	
v2 APPROVED	—issue—>	v2 ISSUED	(PDF generated, validity clock starts)
v2 ISSUED	—customer edit request—>	v3 DRAFT	(v2 → SUPERSEDED)
v2 ISSUED	—accept—>	v2 ACCEPTED	
v2 ISSUED	—expiry sweep—>	v2 EXPIRED	

An issued price is never mutated. That record may be needed in a commercial dispute.

7. Calculation Method Specification

Method	Module	Returns
resolve_rate_card	pricing/rate_resolver.py	Rate card line + which rule matched
calculate_base_freight	pricing/base.py	Base amount honouring charge basis and minimum
apply_weight_breaks	pricing/weight_breaks.py	Best rate including the lower-break check
resolve_applicable_surcharges	pricing/surcharges.py	Surcharge list filtered by mode, port, lane, date
calculate_component	pricing/components.py	One cost line with amount, source, basis
filter_by_incoterm	pricing/incoterm.py	Component codes in scope
build_cost_breakdown	pricing/breakdown.py	Ordered component list + total cost
convert_currency	core/money.py	Decimal conversion with the frozen FX rate
resolve_margin_policy	quotes/margin_policy.py	Floor, target, stretch + which scope matched
apply_margin	quotes/margin_policy.py	Sell price + suppression record if floor bound
evaluate_approval_rules	quotes/approval_rules.py	Required approvers with breach reasons
create_quote_version	quotes/service.py	New immutable version, previous superseded
estimate_win_probability	ml/win_probability/predict.py	Probability + baseline comparison
generate_quote_pdf	quotes/documents.py	Branded PDF with assumptions block

Worked example — build_cost_breakdown

```
build_cost_breakdown(shipment, route, customer) -> CostBreakdown

rate_line, rule = resolve_rate_card(shipment, route, customer)
if rate_line is None:
    rate_line = market_rate_model.predict(...)      # source = PREDICTED

allowed = filter_by_incoterm(shipment.service.incoterm)
components = []

# 1 base
if "OFR" in allowed or "AFR" in allowed:
    base = calculate_base_freight(rate_line, shipment.charge_basis)
    components.append(base)

# 2 freight surcharges (percent-of-base use base.amount)
for sc in resolve_applicable_surcharges(route, shipment, ready_date):
    if sc.code in allowed:
        components.append(calculate_component(sc, base, shipment))

# 3-6 origin, destination, haulage, special handling
... same pattern, each gated on `allowed`

# 7 insurance last - needs freight total
if "INS" in allowed and shipment.additional.needs_insurance:
    freight_total = sum(c.amount for c in components)
    components.append(calculate_insurance(declared_value, freight_total))

total_cost = sum(c.amount for c in components)
return CostBreakdown(components, total_cost, rate_card_rule=rule)
```

8. Data Model Changes

8.1 New collections

Collection	Key fields
rate_cards	carrier_id, type, currency, valid_from, valid_to, status, uploaded_by, source_file_key
rate_card_lines	rate_card_id, lane_key, origin/destination_port_code, container_type, weight_break_min, base_rate {Decimal128, currency}, minimum_charge
surcharges	code, name, calculation_type, value {Decimal128, currency}, applies_to {modes[], ports[], lanes[], container_types[]}, valid_from, valid_to
margin_policies	scope, scope_key, floor_pct, target_pct, stretch_pct, effective_from, is_active
approval_rules	order_index, condition {type, threshold}, approver_role, is_active
quote_approvals	quote_version_id, requested_by, approver_id, approver_role, breach_reason, decision, comment, decided_at
fx_rates	base, quote, rate Decimal128, rate_date, source
rate_card_imports	file_key, status, validation_report, rows_parsed, rows_rejected, committed_at

8.2 Changes to existing collections

Collection	Change
quote_versions	pricing.cost_components[] populated. total_cost, margin_amount, margin_pct, margin_policy_id, margin_suppressed, rate_card_id, rate_card_rule, win_probability filled. is_indicative becomes false. No migration needed — because pricing was designed as a nested object in M1, new keys simply appear on new documents.
freight_quotes	Add requires_approval, approval_status, issued_at, valid_until.
customers	Add credit_profile { status, limit, terms } for approval rule #5.
lane_rates	Deprecated. Keep read-only for one release so M1 quotes remain explainable, then archive.

8.3 Indexes to add

rate_card_lines	{ lane_key: 1, container_type: 1, rate_card_id: 1 }
rate_cards	{ carrier_id: 1, status: 1, valid_from: 1, valid_to: 1 }
surcharges	{ code: 1, valid_from: 1, valid_to: 1, "applies_to.modes": 1 }
margin_policies	{ scope: 1, scope_key: 1, is_active: 1 }
quote_approvals	{ approver_role: 1, decision: 1, created_at: -1 }
fx_rates	{ base: 1, quote: 1, rate_date: -1 }
quote_versions	{ "pricing.margin_pct": 1 } // margin analytics

9. Frontend Build List

Page	New components	Connection notes
/app/quotes/builder/:id	CostBreakdownTable, SurchargeEditor, MarginSlider, MarginFloorWarning, WinProbabilityMeter, RateCardSourceChip	The core screen of this milestone. Margin slider fires PATCH /quotes/{id}/margin on release, not on drag . On 409 QUOTE_BELOW_MARGIN_FLOOR, show the floor warning and reveal "Submit for approval".
/app/quotes/:id	QuoteVersionTimeline, AssumptionsPanel, QuotePdfPreview, ApprovalHistory	Versions are immutable — cache each one indefinitely with staleTime: Infinity. Only the parent quote refetches.
/app/quotes/approvals	ApprovalCard, BreachReasonBadge, RejectDialog	Approver queue. Reject requires a reason; the button stays disabled until text is entered.
/app/pricing/rate-cards	RateCardTable, ImportDropzone, ValidationReport, SupersedeConfirm	Two-step import. The validation report shows row numbers and reasons; commit is disabled while any hard error exists.
/app/pricing/margins	MarginByLaneChart, MarginOutlierTable	Read-only analytics on margin distribution.
/app/admin/margin-policy	PolicyScopeEditor, FloorTargetInputs	Shows the resolution order so an admin can see which policy will win.
/portal/quotes/:id	CustomerQuoteView, AcceptDeclineActions	Customer projection. Cost, margin, buy-rate and win-probability keys stripped in the backend, not hidden in React.

Two frontend rules for this milestone

First: every monetary value stays a **string** from API to display. No `parseFloat`, no `toFixed` outside the shared `<Money />` component. Second: anything under `features/pricing/internal/` must never be imported by a portal page — enforce it with the ESLint `import-boundary` rule, because a careless import is a commercial data leak.

10. Backend Build List

10.1 Modules

Module	Deliverable
pricing/rate_resolver.py	Rate card resolution with the 5-level priority order, returning the matched rule
pricing/base.py	Base freight for all three charge bases with minimum charge
pricing/weight_breaks.py	Break selection including the lower-break optimisation
pricing/surcharges.py	Applicability filtering by mode, port, lane, container type, date
pricing/components.py	All seven calculation types
pricing/incoterm.py	The scope matrix from section 3
pricing/breakdown.py	Ordered assembly, total cost
pricing/importers.py	CSV/XLSX parsing, validation rules, two-phase commit
core/money.py	Decimal arithmetic, rounding policy, FX conversion — the only module allowed to do monetary maths
quotes/margin_policy.py	Resolution order, floor enforcement, suppression recording
quotes/approval_rules.py	Ordered rule evaluation, multi-approver support
quotes/service.py	Version creation, supersede, transaction handling
quotes/documents.py	PDF generation with assumptions and disclaimers
ml/win_probability/	Baseline (historical win rate by lane and tier) plus optional LightGBM
agents/pricing_agent.py	Cost assembly, anomaly flagging, component source labelling
agents/margin_agent.py	Constrained optimisation, sensitivity curve, win probability

10.2 Endpoints

Method	Endpoint	Purpose
GET	/api/v1/pricing/rate-cards	List with validity and status
POST	/api/v1/pricing/rate-cards/validate	Upload → validation report, no write
POST	/api/v1/pricing/rate-cards/commit	Commit a validated import
PATCH	/api/v1/pricing/rate-cards/{id}/status	Activate, expire, supersede
GET/PUT	/api/v1/pricing/surcharges	Surcharge definitions
GET	/api/v1/pricing/cost-breakdown	?shipment_id=&route_id= → itemised components
POST	/api/v1/quotes	Create from shipment + selected route
POST	/api/v1/quotes/{id}/versions	New immutable version
PATCH	/api/v1/quotes/{id}/margin	409 if below floor without authority
POST	/api/v1/quotes/{id}/submit-approval	Route to resolved approvers
POST	/api/v1/quotes/{id}/approve	Approve with comment
POST	/api/v1/quotes/{id}/reject	Reject with mandatory reason
POST	/api/v1/quotes/{id}/issue	PDF, email, start validity timer
GET	/api/v1/quotes/{id}/document	Pre-signed PDF URL
POST	/api/v1/quotes/{id}/accept	Customer acceptance
POST	/api/v1/quotes/{id}/decline	Customer decline with reason
GET	/api/v1/quotes/approvals/queue	Pending approvals with breach reasons
GET/PUT	/api/v1/admin/margin-policies	Floors and targets by scope
GET/PUT	/api/v1/admin/approval-rules	Ordered rule set
POST	/v1/pricing/estimate (agent svc)	Internal: Pricing Agent
POST	/v1/margin/optimize (agent svc)	Internal: Margin Agent

11. Day-by-Day Plan

Day	Backend / Python	Frontend	Data / ML
1	core/money.py with Decimal arithmetic and rounding policy. Precision test suite. Rate card and line models.	CostBreakdownTable and <Money /> against mocked data. ESLint import-boundary rule for pricing/internal.	Obtain at least one real rate card and 150 historical quotes with their component breakdowns. Escalate day 1 if unavailable.
2	Surcharge model with applicability rules. pricing/incoterm.py scope matrix with a unit test per incoterm, including EXW.	Rate card list page. ImportDropzone and ValidationReport against mocks.	Map the historical quotes' components to your component codes. This mapping exercise is where you discover missing surcharges.
3	rate_resolver.py with the 5-level priority order. base.py for all three charge bases.	Quote builder shell: route selection, cost table, component source chips.	Load surcharge values from the rate card and brokerage records. Date-window them correctly.
4	weight_breaks.py including the lower-break rule. surcharges.py applicability filtering.	SurchargeEditor allowing manual add and override, each flagged as MANUAL.	Build the reconciliation harness: run historical quotes through the engine, output per-component deviation.
5	components.py all seven calculation types. breakdown.py ordered assembly. FX conversion with rate freezing.	Cost breakdown wired to the real endpoint. Currency toggle.	First reconciliation run. Expect large deviations. Analyse by component, not by total.
6	margin_policy.py resolution order and floor enforcement with suppression recording. Margin fields on quote versions.	MarginSlider, MarginFloorWarning, live sell price. 409 handling.	Fix the top three component deviations found on day 5. Re-run.
7	approval_rules.py ordered evaluation, multi-approver. quote_approvals model. Approval endpoints.	Approval queue page, ApprovalCard, RejectDialog with mandatory reason.	Win-probability baseline: historical win rate by lane and tier. Compare against a LightGBM model; ship the winner.
8	Quote versioning: immutability, supersede, transaction handling. Issue endpoint. PDF generation.	QuoteVersionTimeline, AssumptionsPanel, PDF preview. Customer portal quote view.	Second full reconciliation. Target $\leq 8\%$ deviation.
9	Rate card two-phase import with all validation rules. Expiry sweep task. Contract tests.	Import flow end to end. Margin analysis page. Empty and error states.	Floor-violation test across every lane $\times$ tier. Timing test for p95.
10	Defect fixing, seed data, permission tests	Playwright: enquiry $\rightarrow$ quote $\rightarrow$ margin adjust $\rightarrow$ approval $\rightarrow$ issue	Evidence pack: reconciliation report, floor-violation output, timing results

The two days that decide this milestone

Day 1 (obtaining a real rate card and historical quotes) and Day 5 (the first reconciliation run). Without real rate data you are building against imagined structures. Without an early reconciliation you discover on day 9 that your engine is 25% off with no time to fix it. Run the reconciliation early even when it is embarrassing — that is what it is for.

12. Definition of Done

12.1 Functional

☐	Item
☐	Cost breakdown shows every component as a named line with its source label
☐	Incoterm correctly includes or excludes each leg; all seven incoterms unit-tested including EXW
☐	Air weight breaks applied, including the lower-break rule, with the chosen break recorded
☐	Rate card resolution follows the 5-level priority and records which rule matched
☐	Missing contract rate falls back to the market model and labels components PREDICTED
☐	Margin floor enforced server-side; below-floor attempts return 409 and create an approval
☐	Agent margin recommendations below floor are suppressed and the suppression recorded
☐	Approval rules evaluate in order; multiple approvals possible on one quote
☐	Quote versions are insert-only; editing an issued quote creates a new version
☐	FX rate, rate card ID, margin policy ID and route snapshot frozen on every version
☐	Rate card import is two-phase; commit blocked while hard errors exist
☐	Branded PDF generated with assumptions block and validity date
☐	Customer portal shows sell price only — no cost, margin or win probability

12.2 Evidence at review

☐	Item
☐	Reconciliation report: $\leq 8\%$ mean absolute deviation across 150 historical quotes, with per-component breakdown
☐	Floor-violation test output: zero violations across every lane $\times$ customer tier
☐	Timing report: $p95 < 60$ seconds under 10 concurrent quote generations
☐	Live demo by someone who did not build it: enquiry $\rightarrow$ priced quote $\rightarrow$ margin below floor $\rightarrow$ approval $\rightarrow$ issue $\rightarrow$ customer view

12.3 Engineering quality

☐	Item
☐	No float anywhere in a calculation path; CI test asserts every monetary field is Decimal128
☐	Monetary values transported as strings; contract test asserts it
☐	Rounding happens in exactly one module with a documented policy
☐	Portal field-leakage test asserts no cost, margin or buy-rate key appears in any customer response
☐	Unit coverage $\geq 85\%$ on pricing, margin policy and approval rules specifically
☐	Immutability test: attempting to update an existing quote version fails
☐	Reproducibility test: a sample of past quotes regenerates identically from frozen inputs

12.4 Carry-forward register

Item	Owner	Target	Blocker